TP1 : L'Évolution des Architectures de Traitement du Langage : Du RNN au Transformer

1. Préparation du Dataset et Vectorisation

1. Charger le dataset IMDB (disponible nativement dans Keras) en limitant le vocabulaire aux **10 000 mots** les plus fréquents.
2. Uniformiser la longueur des séquences à **200 mots max** en utilisant pad_sequences.
3. **Question :** Pourquoi ne peut-on pas donner le texte brut ou des phrases de longueurs différentes directement à nos réseaux de neurones ?

2. Modèle 1 (RNN Simple)

- Créer un modèle contenant :

1. Une couche Embedding (taille vecteur = 64).
2. Une couche SimpleRNN avec 32 unités.
3. Une couche de sortie Dense (1 neurone, activation sigmoid).

3. Modèle 2 (LSTM)

- Créer exactement le même modèle, mais remplacer SimpleRNN par une couche LSTM (32 unités).
- **Analyse :** Entraîner les deux modèles sur 2 epochs. Lequel obtient la meilleure précision ? Pourquoi le RNN simple échoue-t-il sur des textes longs (200 mots) ?

4. Modèle 3 (CNN 1D)

1. Créer un modèle contenant :

1. La couche Embedding.
2. Une couche Conv1D (32 filtres, taille de noyau/kernel = 5, activation relu).
3. Une couche GlobalMaxPooling1D (pour capturer les caractéristiques clés).
4. La couche de sortie Dense.

2. **Analyse :** Comparer le **temps d'entraînement par epoch** du CNN 1D par après celui du LSTM.

5. Modèle 4 (Encoder-Decoder)

1. Créez une architecture composée de :
   1. Un **Encodeur** : une couche LSTM qui traite la phrase et ne renvoie *que* son dernier état caché (return_sequences=False).
   2. Une couche RepeatVector pour dupliquer ce résumé autant de fois qu'il y a de mots dans la cible.
   3. Un **Décodeur** : une seconde couche LSTM qui reconstruit une représentation séquentielle à partir du résumé.
   4. Une couche GlobalMaxPooling1D suivie de la couche de sortie Dense.
2. **Analyse :** En analysant le flux de données entre l'Encodeur et le Décodeur, décrivez la limite majeure de la représentation par un vecteur de contexte fixe lorsque la longueur du texte augmente.

6. **Modèle 5 (Transformer)**
    1. En utilisant l'API fonctionnelle de Keras, implémenter un mini-bloc Transformer :
        1. Connecter la sortie de l'Embedding à une couche MultiHeadAttention (avec num_heads=2 et key_dim=64). L'auto-attention prend le même vecteur en entrée 3 fois (Query, Key, Value).
        2. Ajouter une **connexion résiduelle** (somme de l'entrée et de la sortie de l'attention) suivie d'une LayerNormalization.
        3. Appliquer un GlobalAveragePooling1D puis la couche de sortie Dense.
    2. **Analyse :** Observez les performances finales du Transformer par rapport aux autres modèles.

Partie 2 : Mise en œuvre du mécanisme d'attention des Transformers

Étape 1 : Importations

Importer tensorflow, les composants Keras (Layer, Dense) et numpy.

Étape 2 : Scaled Dot-Product Attention

Créer une classe ScaledDotProductAttention héritant de Layer. Implémenter la méthode call(self, q, k, v, mask=None) :

1. **Scores bruts :** Calculer le produit matriciel entre Q et la transposée de K (utiliser tf.matmul avec transpose_b=True).
2. **Mise à l'échelle :** Diviser les scores par $\sqrt{d_{k}}$, où d_{k} est la taille de la dernière dimension de K (tf.shape(k)[-1]).
3. **Masquage :** Si un masque est fourni, ajouter la valeur (mask * -1e9) aux scores.
4. **Poids :** Appliquer un tf.nn.softmax sur la dernière dimension (axis=-1).
5. **Sortie :** Multiplier les poids d'attention par V via tf.matmul. Retourner la sortie et les poids d'attention.

Étape 3 : Multi-Head Attention

Créer une classe MultiHeadAttention héritant de Layer.

- **__init__(self, d_model, num_heads) :**
    - Vérifier que d_model est divisible par num_heads. Calculez depth = d_model // num_heads.
    - Instanciez 4 couches Dense(d_model) : self.wq, self.wk, self.wv et self.dense.
- **split_heads(self, x, batch_size) :**
    - Redimensionner x en (batch_size, -1, self.num_heads, self.depth) via tf.reshape.
    - Permuter les axes en [0, 2, 1, 3] via tf.transpose.

- **call(self, v, k, q, mask=None) :**
  - Récupérer la taille du batch : batch_size = tf.shape(q)[0].
  - Projeter \(Q\), \(K\), \(V\) via leurs couches Dense respectives et séparer leurs têtes avec split_heads.
  - Passer le tout dans la couche ScaledDotProductAttention.
  - Recombiner les têtes : permuter la sortie en [0, 2, 1, 3] puis fusionner les dimensions en (batch_size, -1, self.d_model) via tf.reshape.
  - Appliquer la couche finale self.dense et retourner la sortie et les poids.

Étape 4 : Validation (Test unitaire)

1. Écrire un script de test avec des tenseurs aléatoires (tf.random.uniform) de dimensions (64, 50, 512) pour \(Q\), \(K\) et \(V\) :

- Tester ScaledDotProductAttention. Vérifier que la forme de la sortie est bien (64, 50, 512).
- Tester MultiHeadAttention avec d_model=512 et num_heads=8. Vérifier que la forme finale reste (64, 50, 512).

2. Refaire le même test de validation que l'étape précédente en utilisant cette fois des données réelles issues de la base IMDB (après vectorisation et passage par une couche d'Embedding). Appliquer le module MultiHeadAttention en mode Auto-Attention (en passant le même tenseur pour V, K et Q) et vérifiez la forme (*shape*) du tenseur obtenu en sortie.